# Git Workflow & Commit Playbook (Prototype 1)

How the team turns the three slices into reviewed, mergeable work — branch model, commit conventions, **when to push and open PRs**, review gates, conflict avoidance, and a 4-week timeline. This operationalizes `docs/repository-structure.md` for Prototype 1.

---

## 1. Branch model

| Branch | Role | Who merges |
|---|---|---|
| `main` | Always stable & demo-ready. Tagged at each client demo. | PR only, after `develop` is green |
| `develop` | Integration branch; all completed slices land here first | PR + 1 review |
| `feature/*` | One per slice/feature (`feature/epic-oauth`, `feature/fhir-mapper`, `feature/supabase-sessions`) | author |

Rules (from `docs/repository-structure.md`): every merge into `develop` or `main` needs a **PR and at least one teammate review**; link issues with `Closes #123`; never commit secrets or `.env`.

```
feature/supabase-sessions ─┐
feature/fhir-mapper ───────┼──► develop ──(demo-ready)──► main ──tag v0.1-proto1
feature/epic-oauth ────────┘
```

### First-time setup (repo currently has no commits)

The working tree is untracked. Land an initial baseline before branching:

```
git add -A
git commit -m "chore: baseline — Demo 1 mock app, docs, supabase config"
git branch develop            # create integration branch
git push -u origin main
git push -u origin develop
```

Then branch every slice off `develop`:

```
git switch develop && git switch -c feature/supabase-sessions
```

---

## 2. Commit conventions (Conventional Commits)

`type(scope): summary` — types: `feat`, `fix`, `chore`, `refactor`, `test`, `docs`. Scopes used in P1: `auth`, `domain`, `services`, `db`, `session`, `ui`.

Principles: - **One logical change per commit** — it should build/typecheck on its own. - Imperative mood, ≤ ~72-char summary, body explains *why* when non-obvious. - Never bundle a dependency bump with feature logic (separate `chore` commit) — keeps lockfile churn reviewable and easy to revert. - Reference issues in the body/footer: `Refs #12`, closing on the PR with `Closes #12`.

---

## 3. When to push (the rule)

Three triggers, in order of importance:

1. **Push the branch on its first commit** — immediately open a **draft PR**. Reasons: off-machine backup, CI runs early, teammates see direction and avoid colliding on the same files.
2. **Push after every green commit** — i.e., it typechecks (`bunx tsc --noEmit`) and any tests pass. Small, frequent pushes shrink merge pain and make review incremental.
3. **Mark the PR "ready for review" only when the slice's Definition of Done is met** and the security checklist (see §5) passes. That's the signal for a teammate to review.

Do **not** wait until a slice is "fully done" to push the first time — that's the most common source of painful conflicts on a 4-person repo.

> Force-push: only on your own un-reviewed `feature/*` branch (e.g., after a local rebase), never on `develop` / `main`, never after someone has reviewed/based work on your branch.

---

## 4. Consolidated commit timeline (all three slices)

Each slice's full commit list lives in its doc (01 §8, 02 §9, 03 §8). Summary of push/PR moments:

| Slice / branch | First push (draft PR) | Ready-for-review | Merges to develop |
|---|---|---|---|
| `feature/supabase-sessions` | commit 1 (`chore(db)`) | commit 7 (`docs(db)`) | **first** |
| `feature/fhir-mapper` | commit 1 (`feat(domain): GameContext`) | commit 8 (`wire real path`) | **second** |
| `feature/epic-oauth` | commit 1 (`chore(auth): deps`) | commit 7 (`docs(auth)`) | **third** |

Why this merge order (also in Doc 00 §6): Supabase has no UI dependency (land it first); the mapper defines the `GameContext` seam the auth slice plugs into (second); auth is hardest to verify and depends on the seam (last). The mock default keeps `develop` demoable at every step.

---

## 5. PR template & review gates

Open every PR (even drafts) with this body so reviews are consistent. The security questions come straight from `docs/security-and-scope.md` §"PR review reminders."

```
## What & why
<one paragraph; link the slice doc>

## Slice / scope
Closes #__   ·   Slice: A/B/C

## Checklist
- [ ] Typechecks (`bunx tsc --noEmit`) and tests pass
- [ ] One logical change per commit; Conventional Commit messages
- [ ] No secrets / tokens / .env committed
- [ ] Touches auth/FHIR/AI? → secrets stay server-side (anon key / SecureStore only)
- [ ] No log prints tokens or full FHIR bundles
- [ ] UI copy implies no medical advice; sandbox badge intact
- [ ] In scope for Prototype 1 (no quiz/LLM/extra tables)

## How to test
<commands / device steps / verification query>
```

**Merge gate:** 1 teammate approval + green checklist. Security-touching PRs (all three P1 slices touch auth, FHIR, or DB) should get the reviewer to explicitly tick the secret-handling boxes.

---

## 6. Avoiding conflicts across parallel slices

The slices were scoped to **disjoint files** so three people can work at once:

| Slice | Owns (no one else edits) | Shared file — coordinate |
|---|---|---|
| A | `services/epicAuth`, `services/secureTokens`, `features/auth`, `app/connect-epic` | `App.tsx` (add AuthProvider), `nav.tsx` (add ScreenName) |
| B | `domain/gameContext`, `domain/fhirMapper`, `domain/dataSource`, `services/fhir`, `__fixtures__` | `app/patient-snapshot.tsx` (import seam) |
| C | `supabase/migrations`, `services/supabase`, `features/session` | — |

Shared-file rules: - `App.tsx`, `nav.tsx`, `patient-snapshot.tsx` are touched by A and B. Land **B's seam edit first** (it merges before A), so A rebases onto a `develop` that already has the seam. - Rebase your feature branch on `develop` before opening "ready for review": `git switch feature/x && git fetch && git rebase origin/develop`. - Keep `package.json` / `bun.lock` changes in their own `chore` commits to localize lockfile conflicts.

---

## 7. Four-week timeline (maps commits to weeks)

Aligned to `docs/prototype-phases.md` (P1 = weeks 1–4) and the meeting cadence (client/GTA Thursdays, internal Fridays).

| Week | Goal | Branches active | Key commits / merges | Demo checkpoint |
|---|---|---|---|---|
| **1** | Foundations + unblock | baseline, `feature/supabase-sessions` | baseline commit; C commits 1–5; **register Epic app & redirect URI**; raise open questions #1,#2,#3,#7 at Thu meeting | Friday: `sessions` table pushes; row insert works locally |
| **2** | Seam + data shape | merge C → develop; `feature/fhir-mapper` | B commits 1–4 (GameContext, dataSource, snapshot reads seam, fixtures) | Thu: show seam (mock still default), session row in dashboard |
| **3** | Real data + auth | merge B → develop; `feature/epic-oauth` | B commits 5–8 (fhir.ts, mapper, tests, real path); A commits 1–5 | Thu: mapper tests green; auth login on a device |
| **4** | Integrate + harden | merge A → develop → `main` | A commits 6–7; rebases; PHI checklist; tag `v0.1-proto1` | Thu demo: connect Epic → real meds/conditions → session row created |

Buffer: keep Week 4's back half for bug-fix-only (no new features), matching the plan's stabilization posture.

---

## 8. Secrets & .gitignore (do this in Week 1)

- Confirm `apps/mobile/.gitignore` (and/or root) ignores `.env`, `*.env`, and Expo build artifacts. Verify with `git check-ignore -v apps/mobile/.env` — it must report a match before anyone creates their `.env`.
- Only `.env.example` is tracked. Epic client secret (if confidential) and OpenRouter keys live in **Supabase project secrets**, never in git, never in `EXPO_PUBLIC_*`.
- Before each Thursday demo, run the PHI/sandbox checklist in `docs/security-and-scope.md` (Summer owns sign-off): sandbox URLs only, no real identifiers in DB/logs/screenshots, no keys in the mobile bundle.

---

## 9. Quick reference — daily loop

```
git switch develop && git pull                  # start from latest integration
git switch -c feature/<slice>                   # or switch back to yours
#  ... edit one logical change ...
bunx tsc --noEmit                               # typecheck (and run tests if present)
git add -p && git commit -m "feat(scope): ..." # one logical change
git push                                         # first push → open draft PR
# when slice DoD + security checklist pass:
git fetch && git rebase origin/develop          # resolve conflicts locally
git push --force-with-lease                      # update your own branch
# mark PR ready → get 1 review → squash/merge to develop
```

That loop, applied per slice in the C → B → A order, lands all of Prototype 1 on `develop`, then a single reviewed PR promotes `develop` → `main` for the Week 4 demo tag.

---